

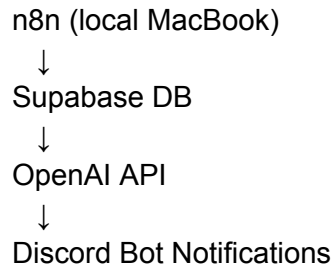
Week 1 Goal

By the end of week 1, you should have:

- A dedicated Mac mini-server setup
- n8n running locally
- Supabase connected
- OpenAI API working
- Discord bot working
- A basic database schema
- One working end-to-end workflow:
 - fake/sample job enters system
 - stored in database
 - AI analyzes it
 - Discord notification sent

If you complete ONLY this, you already have the skeleton of the entire project.

Week 1 Architecture



PHASE 1 — Dedicated Machine Setup

Step 1 — Prepare The MacBook

Requirements:

- Always plugged in
- Never sleeps

- Separate browser profile
- No personal accounts
- Stable internet

Mac settings:

- Disable sleep
- Disable auto updates temporarily
- Install:
 - Homebrew - package manager for MAC OS
 - Docker Desktop - lets me package dependencies and stuff into a container
 - [Node.js](#) - n8n is built on this
 - VS Code

Commands:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"  
brew install node
```

Install:

- Docker Desktop
- VS Code manually

PHASE 2 — Create Core Accounts

Step 2 — Create Services

Create accounts for:

- [Supabase](#)
- [OpenAI Platform](#)
- [Discord Developer Portal](#)
- [n8n Cloud docs/reference](#)

Keep all API keys in:

~/career-agent/.env

PHASE 3 — Install n8n

Step 3 — Run n8n Locally

Recommended:

Docker.

Create folder:

```
mkdir ~/career-agent  
cd ~/career-agent
```

Create:

docker-compose.yml

Contents:

version: '3'

services:

n8n:

image: n8nio/n8n

ports:

- "5678:5678"

environment:

- N8N_HOST=localhost

- N8N_PORT=5678

- N8N_PROTOCOL=http

volumes:

- ./n8n_data:/home/node/.n8n

Run:

```
docker compose up -d
```

Open:

<http://localhost:5678>

Create admin account.

PHASE 4 — Supabase Setup

Step 4 — Create Database

Inside Supabase:

Create project.

Create tables:

jobs

```
create table jobs (  
  id bigint generated always as identity primary key,  
  title text,  
  company text,  
  location text,  
  description text,  
  salary text,  
  url text,  
  score integer,  
  created_at timestamp default now()  
);
```

contacts

```
create table contacts (  
  id bigint generated always as identity primary key,  
  name text,  
  company text,  
  role text,  
  notes text,  
  followup_date date,  
  created_at timestamp default now()  
);
```

applications

```
create table applications (  
  id bigint generated always as identity primary key,  
  company text,  
  role text,  
  status text,
```

```
    applied_date date,  
    notes text  
);
```

Get:

- Supabase URL
- Supabase anon key

Store in `.env`.

PHASE 5 — OpenAI API

Step 5 — Verify GPT Works

Create API key.

Set low spending limit:

- \$5–10 initially

Test with Python:

```
from openai import OpenAI
```

```
client = OpenAI(api_key="YOUR_KEY")
```

```
response = client.chat.completions.create(  
    model="gpt-4.1-mini",  
    messages=[  
        {"role": "user", "content": "Say hello"}  
    ]  
)
```

```
print(response.choices[0].message.content)
```

Goal:

Verify API works.

PHASE 6 — Discord Bot

Step 6 — Create Notification System

In Discord Developer Portal:

- Create application
- Add bot
- Enable message permissions

Invite bot to your private server.

Copy bot token.

PHASE 7 — First Real Workflow

THIS is the most important part of week 1.

Step 7 — Build End-to-End Workflow In n8n

Workflow:

Manual Trigger



Fake Job Data



OpenAI Job Analysis



Store In Supabase



Send Discord Notification

Example Fake Job

```
{  
  "title": "Embedded Software Engineer Intern",  
  "company": "Lockheed Martin",  
  "location": "Atlanta, GA",  
  "description": "C++, embedded systems, AI autonomy work"  
}
```

Step 8 — GPT Scoring Prompt

Prompt:

You are evaluating jobs for a computer engineering student interested in AI, embedded systems, defense, and high-income career growth.

Score this job from 1-100.

Return ONLY JSON:

```
{  
  "score": number,  
  "reason": "short explanation"  
}
```

Store:

- score
- reasoning

in Supabase.

Step 9 — Discord Notification

Example message:

New High-Match Job Found

Lockheed Martin
Embedded Software Engineer Intern
Atlanta, GA

Match Score: 84

Reason:

Strong embedded systems + defense alignment.

Once THIS works:

you officially have an AI-enhanced automation pipeline.

PHASE 8 — Organize The Repo

Step 10 — GitHub Repository

Structure:

```
career-agent/  
├── docker-compose.yml  
├── workflows/  
├── prompts/  
├── scripts/  
├── docs/  
├── README.md  
└── .env
```

Commit EVERYTHING cleanly.

IMPORTANT RULES FOR WEEK 1

Rule 1

Do NOT optimize architecture.

Rule 2

Do NOT build frontend yet.

Rule 3

Do NOT touch LinkedIn yet.

Rule 4

Do NOT build agents yet.

Rule 5

Focus ONLY on:

- data flow
 - reliability
 - integrations
-

Stretch Goals (ONLY if ahead)

Optional:

- Telegram instead of Discord
- scheduled workflow
- real public jobs API
- basic dashboard

But only after end-to-end flow works.

What You Learn In Week 1

By the end of week 1 you will understand:

- APIs
- databases
- automation pipelines
- AI integration
- workflow orchestration
- event-driven systems
- basic backend architecture

This is the foundation for the entire project.